Cyprus University of Technology

# CSE 226
# Algorithms and Complexity

**Lecture 13**
**Minimum Spanning Trees (Part 2)**
**Ελαφρύτατα Συνδετικά Δένδρα (Μέρος 2)**

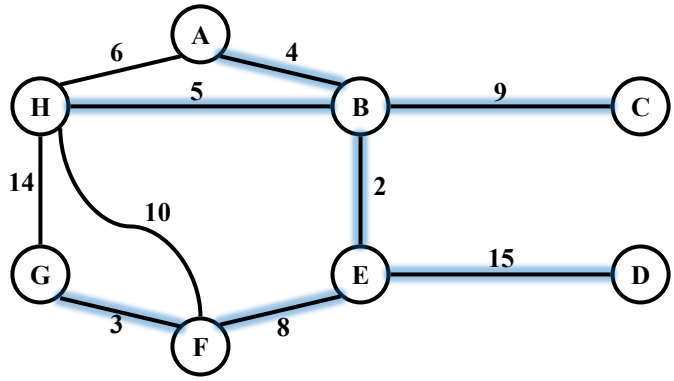
Herodotos Herodotou

*Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis*



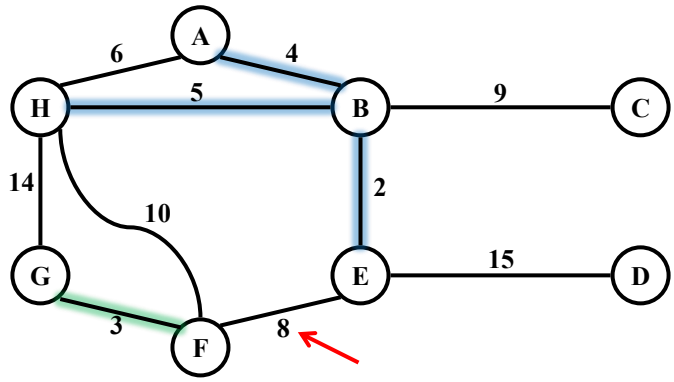
## Minimum Spanning Tree

- **Problem:** given a connected, undirected, weighted graph, find a **spanning tree** (**συνδετικό δέντρο**) $T$ using edges that *minimize* the total weight, $w(T) = \sum_{(u,v)\in T} w(u,v)$



## Kruskal's Algorithm

- Starts with each vertex being its own component
- Repeatedly merges two components into one by choosing the **light edge** (**ελαφρά ακμή**) that connects them





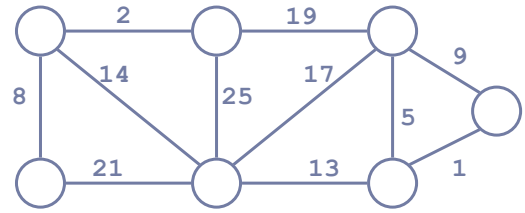



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T ∪ {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

## Kruskal's Algorithm

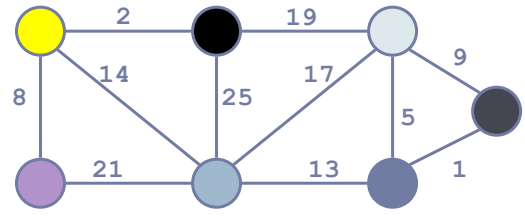
*Run the algorithm:*




```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

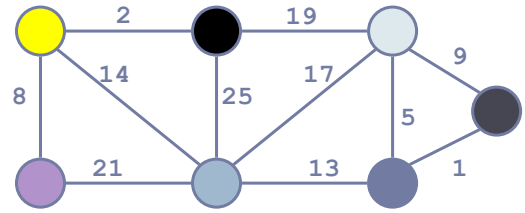


## Kruskal's Algorithm




```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

## Kruskal's Algorithm

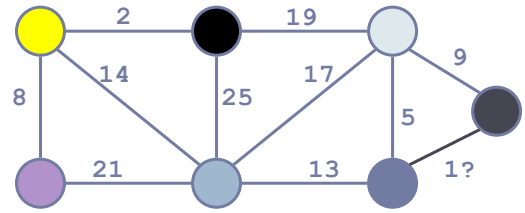



```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```





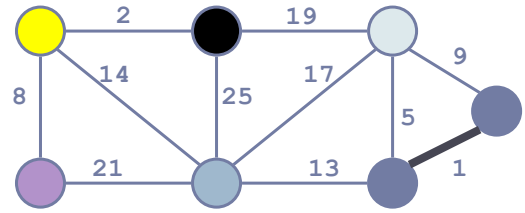
## Kruskal's Algorithm




```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

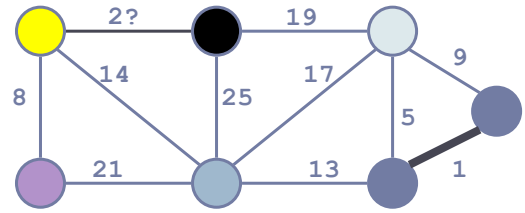






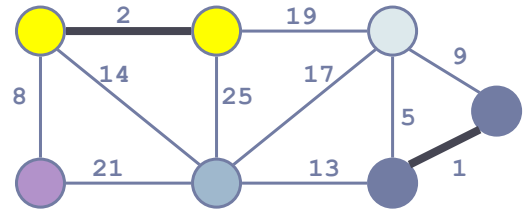

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

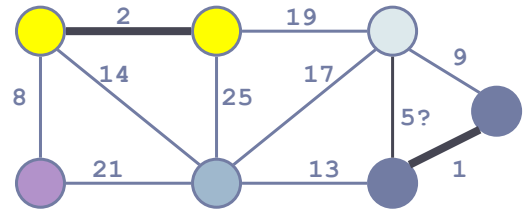






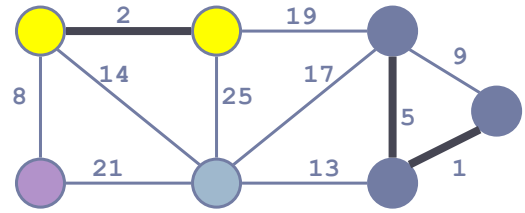

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

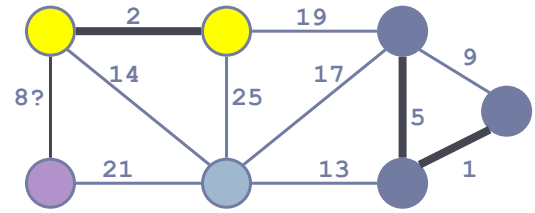
```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```





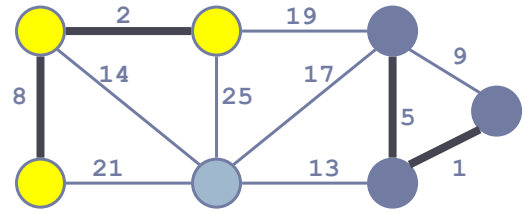

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

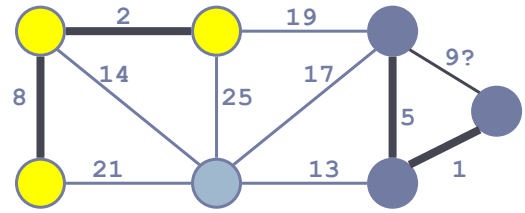
```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```





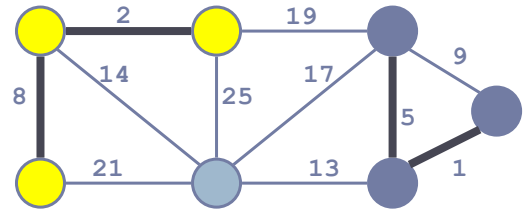

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

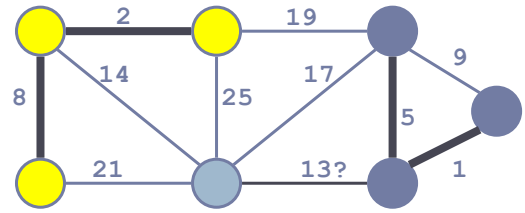






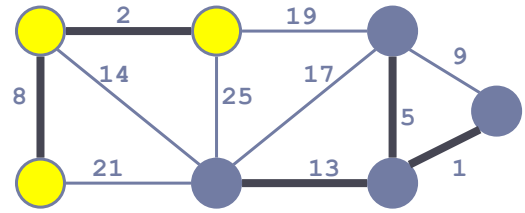

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

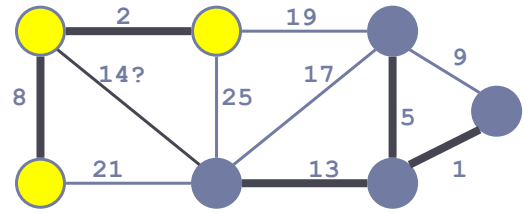






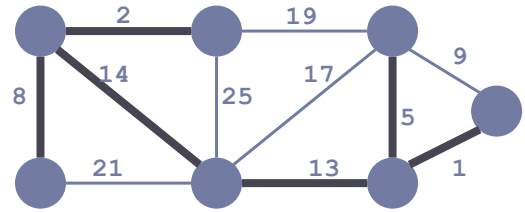

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

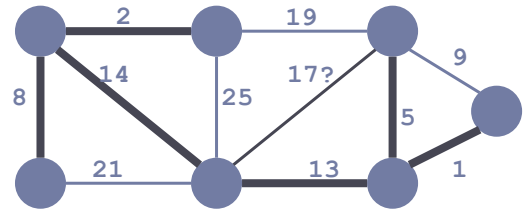
```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```





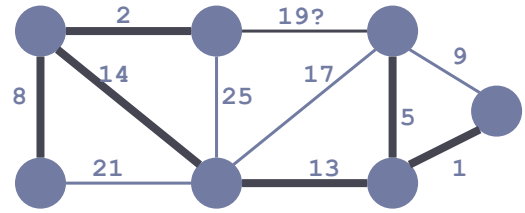

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

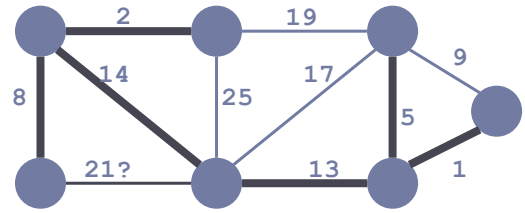






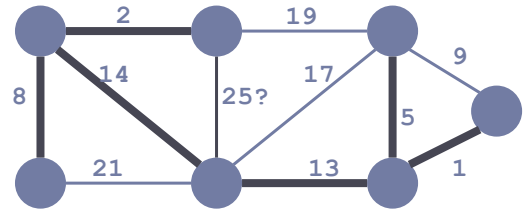

## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```

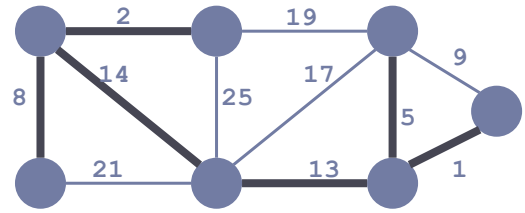







## Kruskal's Algorithm

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);
   sort G.E by increasing edge weight w
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {(u,v)};
         Union(FindSet(u), FindSet(v));
}
```



## Kruskal's Algorithm

*What will affect the running time?*

```
Kruskal(G, w)
{
   T = ∅;
   for each v ∈ G.V
      MakeSet(v);        ← O(V) calls
   sort G.E by increasing edge weight w   ← 1 sort
   for each (u,v) ∈ G.E (in sorted order)
      if FindSet(u) ≠ FindSet(v)   ← O(E) calls
         T = T ∪ {(u,v)};
         Union(FindSet(u), FindSet(v));   ← O(V) calls
}
```

*How many exactly?* $V - 1$





## Kruskal's Algorithm: Running Time

- To summarize:
  - Sort edges: $O(E \lg E)$
  - $O(V)$ MakeSet()'s
  - $O(E)$ FindSet()'s
  - $O(V)$ Union()'s
- Upshot:
  - Best **disjoint sets** (**ξένα σύνολα**) algorithm makes each operation listed above take $O(\alpha(V))$
    - $\alpha$ very slowly growing function
  - Total: $O(E \lg E) + O((V + E)\alpha(V))$
  - But: $\alpha(|V|) = O(\lg V) = O(\lg E)$
  - Overall: $\boldsymbol{O(E \lg E) \equiv O(E \lg V)}$

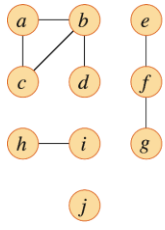
## Disjoint Sets

- Want a data structure to support **disjoint sets** (**ξένα σύνολα**)
  - Collection of disjoint sets $S = \{S_i\}, S_i \cap S_j = \emptyset$
- Need to support following operations:
  - MakeSet($x$): $S = S \cup \{x\}$
  - Union($S_i, S_j$): $S = S - S_i - S_j \cup \{S_i \cup S_j\}$
  - FindSet($x$): return $S_i \in S$ such that $x \in S_i$





## Disjoint Sets: Example



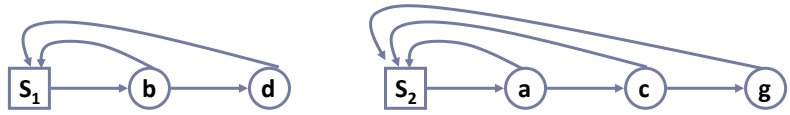

(a)

| Edge processed | Collection of disjoint sets | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| initial sets | $\{a\}$ | $\{b\}$ | $\{c\}$ | $\{d\}$ | $\{e\}$ | $\{f\}$ | $\{g\}$ | $\{h\}$ | $\{i\}$ | $\{j\}$ |
| $(b, d)$ | $\{a\}$ | $\{b, d\}$ | $\{c\}$ | | $\{e\}$ | $\{f\}$ | $\{g\}$ | $\{h\}$ | $\{i\}$ | $\{j\}$ |
| $(e, f)$ | $\{a\}$ | $\{b, d\}$ | $\{c\}$ | | $\{e, f\}$ | | $\{g\}$ | $\{h\}$ | $\{i\}$ | $\{j\}$ |
| $(a, c)$ | $\{a, c\}$ | $\{b, d\}$ | | | $\{e, f\}$ | | $\{g\}$ | $\{h\}$ | $\{i\}$ | $\{j\}$ |
| $(h, i)$ | $\{a, c\}$ | $\{b, d\}$ | | | $\{e, f\}$ | | $\{g\}$ | $\{h, i\}$ | | $\{j\}$ |
| $(a, b)$ | $\{a, b, c, d\}$ | | | | $\{e, f\}$ | | $\{g\}$ | $\{h, i\}$ | | $\{j\}$ |
| $(f, g)$ | $\{a, b, c, d\}$ | | | | $\{e, f, g\}$ | | | $\{h, i\}$ | | $\{j\}$ |
| $(b, c)$ | $\{a, b, c, d\}$ | | | | $\{e, f, g\}$ | | | $\{h, i\}$ | | $\{j\}$ |

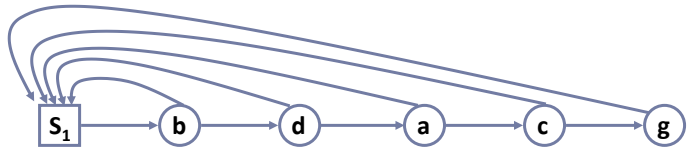
(b)

## Disjoint Sets: Implementation

- So how do we implement disjoint sets?
  - Naïve implementation: use a **linked list** to represent each set:




  - Union$(S_1, S_2)$

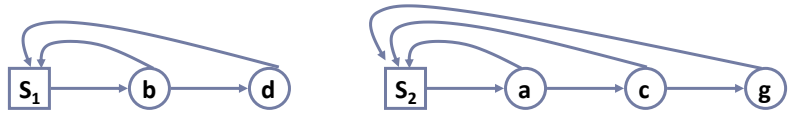







## Disjoint Sets: Implementation

- So how do we implement disjoint sets?
  - Naïve implementation: use a **linked list** to represent each set:




  - Running times
    - MakeSet(): create a singleton list => $O(1)$
    - FindSet(): return pointer to list's head => $O(1)$
    - Union$(S_i, S_j)$: append elements of $S_j$ into $S_i$ => $O(S_j)$
      - *How long can a single Union() call take?*
      - *How long will $n$ Union() calls take?*

## Disjoint Sets: Analysis

- Worst-case analysis: $O(n^2)$ time for $n$ Unions

| | |
|---|---|
| Union$(S_2\ S_1)$ | append 1 element |
| Union$(S_3, S_2)$ | append 2 elements |
| Union$(S_4, S_3)$ | append 3 elements |
| … | |
| Union$(S_n, S_{n-1})$ | append $n-1$ elements |
| | $O(n^2)$ |

- **Improvement:** always append smaller list into larger
  - *Why will this make things better?*
  - *What is the worst-case time of Union()?*
- But now $n$ Unions take only $\mathbf{O(n \lg n)}$ time!



## Disjoint Sets: Amortized Analysis

- **Amortized analysis** (**αντισταθμιστική ανάλυση**) computes average times without using probability
- With our new Union(), **any individual element $x$** is "copied" at most $\lg n$ times when forming the complete set from single-element sets
- We say that each Union() takes $\mathbf{O(\lg n)}$ ***amortized time***

| Time when $x$ is appended | Size of resulting set |
|---|---|
| 1st | ≥ 2 |
| 2nd | ≥ 4 |
| 3rd | ≥ 8 |
| … | … |
| $k$th | ≥ $2^k$ |
| … | … |
| $\lg n$ | ≥ $n$ |